

Reinforcement Learning for Robotic Control: Sim-to-Real Transfer via Domain Randomization

Gianbattista Busonera
Politecnico di Torino
s353805

Alessandro de Stasi
Politecnico di Torino
s354862

Gabriele Tebano
Politecnico di Torino
s357952

Giovanni Casati
Politecnico di Torino
s354621

Abstract

This report examines reinforcement learning for robotic control and sim-to-real transfer in a controlled sim-to-sim setting. We implement REINFORCE (using three different baselines) and Actor-Critic from scratch for the Hopper task. After that, we assess PPO and SAC using Stable-Baselines3 on the PandaGym PushTask. We use Uniform Domain Randomization (UDR), Automatic Domain Randomization (ADR) and Bounded Automatic Domain Randomization (BADR) to reduce the 4 kg mass domain gap. The results highlight that the adaptive baseline helps to stabilize REINFORCE learning, Actor-Critic achieves the best hopping behavior, SAC outperforms PPO and simpler domain randomization strategies are more appropriate for the PushTask. [GitHub repository](#)

1. Introduction

Reinforcement Learning (RL) is an effective approach for learning control policies in robotics, allowing agents to learn complex behaviours through interaction with their environment [10]. A major challenge in real-world deployment is the *sim-to-real* gap: policies trained in simulation often do not perform as well when applied to physical hardware [4]. This project looks into the sim-to-real issue in a controlled *sim-to-sim* environment, where the difference in reality is reproduced by varying a single physical parameter (the cube’s mass) between a *source* training domain and a *target* test domain. Our work is divided into three sections.

Firstly, we implement REINFORCE [14] and Actor-Critic [5] from scratch and analyse their learning dynamics on the Hopper task.

Secondly, we establish baseline performance on the PushTask using PPO [9] and SAC [3] via Stable-Baselines3 [7], characterising the sim-to-real degradation.

Thirdly, we implement UDR [12] and ADR [6] to reduce the sim-to-sim gap. We observe that canonical ADR encounters limitations in this task, and propose a simple

bounded variant (BADR) to address them.

2. Methodology

2.1. Gym Hopper Environment

A flat, one-legged robot present in MuJoCo [1, 13]. The state space is continuous represented by $\mathbb{R}^{11}$ which includes joint positions, velocities and the height and angle of the torso. The action space is continuous, represented in $\mathbb{R}^3$ with one torque per hinge joint. The reward promotes moving forward while discouraging excessive control effort. It also provides a positive bonus for staying in a healthy, upright posture. An episode ends when the agent goes beyond the allowed state limits or when it hits 1 000 timesteps. The evaluation metric is the mean cumulative undiscounted episodic return over 50 episodes.

2.2. PandaGym PushTask Environment

The PandaGym [2] PushTask requires a 7-DoF Franka Emika Panda manipulator to push a cube to a target position on a table.

The state space is continuous ($\mathbb{R}^{25}$), divided into **observation**, describing the gripper’s position and velocity, along with the cube’s position, orientation, linear velocity, rotational velocity, and **desired goal** describing the x, y, z coordinates of the target location for the cube. The action space is continuous ($[-1, 1]^7$) and each entry of the 7-dimensional vector describing an action refers to the normalized angular velocity applied by each of the robot’s joints. The reward system can either be sparse (-1 or 0 , based on whether the gripper successfully moves the cube within a specified threshold w.r.t. the goal position) or dense (at each time step the agent receives a reward equal to the negative distance between the current cube’s position and the target one). In our work we will use a **dense** reward.

The source domain uses a cube mass $m_s = 1$ kg, while the target domain uses $m_t = 5$ kg, introducing a $+4$ kg shift that the agent must generalize across. The observed metrics for evaluation are the success rate and mean return ($\bar{r}$) (with

its std) of the model over 50 episodes, along with

$$\text{gap closed} = \frac{\bar{r} - \bar{r}_{lb}}{\bar{r}_{ub} - \bar{r}_{lb}},$$

defined as the fraction of the upper-lower bound gap on mean return covered by each domain randomization method. The upper and lower bounds are the performance of models trained respectively on the target and source domain.

2.3. REINFORCE

REINFORCE is a Monte Carlo policy-gradient method [14] updating after each episode using:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - b) \right], \quad (1)$$

where $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$ is the discounted return from timestep t , and b is a baseline used to reduce gradient variance without introducing bias [10]. The policy is a Gaussian MLP with two hidden layers of 64 units and $\tanh$ activations justified by the action space support. It produces the mean of the action distribution, while the standard deviation is a single learnable scalar parameter (passed through a softplus to ensure positivity) that is independent of the state.

We compare three baseline strategies:

(1) **Vanilla** ($b = 0$), (2) **Constant** ($b = 20$), (3) **Adaptive** ($b = 25$ —th percentile of the last 100 initial returns G_0): instead of using a fixed value, the algorithm compares the current episode to the policy’s recent performance approximating the stabilizing effect of a critic. The most relevant hyperparameters are: learning rate $\alpha = 10^{-3}$, discount factor $\gamma = 0.99$ and no reward normalization.

2.4. Actor-Critic

Actor-Critic replaces the high-variance Monte Carlo return with a bootstrapped advantage estimate [10]:

$$A_t = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t), \quad (2)$$

where V_{ϕ} is a neural network (critic) with the same architecture as the policy (kept identical to REINFORCE), only differing in the output (scalar instead of vector). Actor and critic are trained simultaneously and updated at each time step; the critic minimizes the mean squared error $\mathcal{L}_c = \mathbb{E}[(A_t)^2]$, while the actor updates the weights using the following rule [11]

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \log \pi_{\theta}(a|s) A_t. \quad (3)$$

Action sampling, standard deviation learning and discount factor value are kept identical to the ones in REINFORCE. For reasons that will be explained later, we change the learning rate to $\alpha = 2.5 \cdot 10^{-4}$.

NOTE: actor and critic network share the same optimizer (Adam), hence the learning rate is equal for both networks.

2.5. PPO and SAC (Stable-Baselines3)

We use the Stable-Baselines3 library implementation[7] of the PPO and SAC algorithms to train an agent on the Push-Task. Proximal Policy Optimization [9], originally developed at OpenAI, is an on-policy algorithm that aims to constrain policy updates through a clipped surrogate objective function, preventing excessively large updates that could destabilize training. The objective function becomes:

$$L_t^{\text{PPO}} = \min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)$$

with the ratio term r_t defined as: $r_t = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$

Soft Actor-Critic [3], originally developed at Berkeley, is an off-policy algorithm that introduces an entropy term in the objective function, encouraging exploration and helping the agent escape sub-optimal strategies.

Namely, the objective function becomes:

$$L^{\text{SAC}} = \mathbb{E}_{\pi} \left[\sum_t \gamma^t (R_t + \alpha \mathcal{H}(\pi(\cdot | s_t))) \right]$$

where $\mathcal{H}$ denotes the entropy of the policy.

Various hyperparameters configurations were tested for both PPO and SAC. Among the two algorithms, SAC consistently achieved better results and more stable performances. Table 1 summarizes the main SAC hyperparameters used during training. All other parameters were left to default. The entropy coefficient was set to 'auto' making it a learnable parameter during training to ensure the policy’s entropy stayed sufficiently high, we used 4 parallel environments to fully exploit the GPU, accelerate training and produce a more diverse replay buffer for policy updates.

Table 1. main SAC training hyperparameters

Parameter	Value	Parameter	Value
Learning rate	0.0003	Gradient steps	4
Batch size	256	Learning starts	10,000
Training timesteps	600,000	Entropy coeff.	auto
Replay buffer size	600,000	Discount (γ)	0.98

2.6. Domain Randomization

For the current task we randomize a single dynamics parameter, the cube mass m , changed after every training episode.

2.6.1. Uniform Domain Randomization (UDR)

At the beginning of each episode the mass is sampled uniformly from a provided range:

$$m \sim \mathcal{U}(m_{\min}, m_{\max}). \quad (4)$$

After trying various intervals we settled on $[m_{\min}, m_{\max}] = [0.5, 10]$ kg: a range that includes the target mass (5 kg)

without relying on knowledge of it, and that largely spans both heavier and lighter objects than the source (1 kg).

2.6.2. Automatic Domain Randomization (ADR)

ADR [6] treats the randomization bounds $[m_{\min}, m_{\max}]$ as learnable parameters: it starts from a collapsed range and expands or contracts each extreme based on the agent’s performance when sampled at that boundary. The mass is sampled at reset according to a probability of boundary sampling $p_b = 0.8$:

$$\begin{cases} m \sim \mathcal{U}(m_{\min}, m_{\max}) & \text{with } p = 1 - p_b \\ m = m_{\min} & \text{with } p = p_b/2 \\ m = m_{\max} & \text{with } p = p_b/2 \end{cases} \quad (5)$$

Performance buffers of size $K = 20$ accumulate the episodic return for each boundary; a performance threshold τ determines whether boundaries are expanded or contracted by a value $\Delta = 0.2$:

- If mean return $> \tau_{\text{high}}$: expand boundaries.
- If mean return $< \tau_{\text{low}}$: contract boundaries.

We set $\tau_{\text{high}} = -10$ and $\tau_{\text{low}} = -25$. We prevent the lower extreme from becoming smaller than (10^{-3} kg) to keep the mass strictly positive, but impose no upper limit. The starting point for ADR corresponds with the midpoint of the UDR range, to allow for fair comparison between the two strategies and meaningful boundary expansion analysis.

2.6.3. Bounded ADR (BADR)

Canonical ADR on this task presents a weakness: the upper bound expands beyond any reasonable cube mass. Our Bounded ADR (BADR) variant introduces two changes:

1. the supplied mass range is reinterpreted as a hard limit $[m_{\min}^{\text{cap}}, m_{\max}^{\text{cap}}]$; the range can never grow beyond $[0.5, 10] \text{ kg}$ in our setup (same range as UDR);
2. once an extreme reaches its cap value it is marked as *saturated*: boundary sampling on that side stops and no further expansion or retraction occurs. With one side saturated the entire p_b acts on the remaining one; when both sides saturate BADR degenerates to normal UDR over the full range.

All other ADR hyper-parameters are left unchanged. BADR is therefore in between UDR and ADR; starting at a single point and expanding outwards, it aims to:

- avoid the disruptive early updates UDR can suffer when masses are drawn from opposite ends of a wide range;
- prevent the unbounded expansion of canonical ADR, which may otherwise drift toward masses not meaningful in the task’s context.

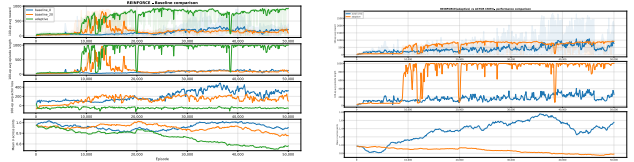
3. Experiments

3.1. Part 1 – Hopper Environment

3.1.1. REINFORCE

We trained three REINFORCE variants for 50,000 episodes using a fixed random seed. Fig. 1a shows the training evolution through 100-episode rolling averages of cumulative episodic rewards, episode length, actor loss, and mean policy standard deviation across joints.

The adaptive baseline ($b=P_{25}$) achieved the highest peak reward (941) and recovered after a performance drop around episode 21k. The constant baseline ($b=20$) reached its peak reward earlier (13.1k episodes) but collapsed around episode 14k without recovery. The vanilla variant ($b=0$) obtained the lowest peak reward (330) and required more episodes to reach peak performance.



(a) REINFORCE variants training (b) Actor-Critic vs. best REINFORCE behaviour on Hopper-v4.

Figure 1. Training dynamics on Hopper-v4.

3.1.2. Actor-Critic

Training strategy is identical to REINFORCE. The early phase (ep 100 - 600) is noisy and with low reward. Next, the model slowly learns in an unstable way (avg reward 250 - 600). At episode 45 200 we reach a peak of **1097**; some episodes later the reward drops back to 400-500 range. On average, episode length is much lower than REINFORCE, despite reaching the highest cumulative reward among all models. This shows that the agent learns to hop forward, bringing more cumulative reward despite falling earlier. Moreover, the graph shows a constant decrease of REINFORCE’s std throughout the training, whereas Actor critic’s std rises multiple times.

Table 2. **Evaluation** results on Hopper-v4. Actions are deterministic (mean value is used without sampling). Results are consistent with the observed training dynamics in 3.1.1 and 3.1.2

Metric	REINFORCE			Actor-Critic
	$b=0$	$b=20$	$b=P_{25}$	
Episodic mean return (50 ep.)	305.03	856.46	962.07	1157.30

3.2. Part 2 – PushTask and Domain Randomization

3.2.1. Sim-to-Sim Baselines

The model proved to be quite robust to the environment change: even when trained with a mass of 1 kg, it still obtained a success rate very close to 100% on the target environment, where the mass is slightly higher.

Table 3 reports the mean return over 50 test episodes, along with the success rate for the three train→eval configurations. Figure 2 compares more comprehensively the agent trained on the source environment with models trained using domain randomization techniques on environments with a variety of masses.

Table 3. Baseline results on PushTask (50 episodes)

Configuration	Mean return	Std	Success rate
source → source	−0.570	0.387	100%
source → target	−0.696	0.709	96%
target → target	−0.465	0.286	100%

3.2.2. Domain Randomization

Table 4 summarizes the target-domain performance of the three randomization strategies, against the reference bounds of Table 3.

Table 4. DR results on PushTask, target domain ($m = 5$ kg, mean $\pm$ std, 50 episodes).

Method	Mean return	Success rate	Gap cl.
Lower bound (source→target)	−0.696 $\pm$ 0.709	96%	–
UDR	−0.511 $\pm$ 0.310	100%	80%
ADR	−0.785 $\pm$ 0.755	96%	−38%
BADR	−0.545 $\pm$ 0.389	100%	66%
Upper bound (target→target)	−0.465 $\pm$ 0.286	100%	–

The **UDR** wide manually-chosen range $[0.5, 10]$ kg already closes 80% of the gap between the source-only baseline and the target-trained upper bound, with success rate equal to 1.00 and std being less than half the one of the source-only model. Canonical **ADR** fails on this task, being the worst performing model and having the highest standard deviation. **BADR** addresses the failure of ADR by capping the range at $[0.5, 10]$ kg. By the end of training, both sides have saturated and sampling becomes equivalent to UDR. The final policy closes 66% of the gap, with success rate at 100% but does not outperform UDR.

Out-of-distribution sweep. Fig. 2 shows results for each policy beyond the single target mass of 5kg.

Success rate’s feedback is limited: every method obtains $\geq 92\%$ success rate. Mean return is more discriminative

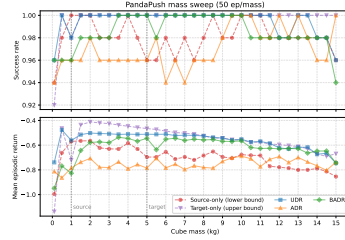


Figure 2. PandaPush mass sweep, 50 episodes per mass. Top: success rate. Bottom: mean episodic return. Vertical dotted lines mark the source (1 kg) and target (5 kg) masses.

and confirms Table 4 ranking: UDR and upper-bound perform similarly; BADR places just below; source-only is consistently lower and ADR is the noisiest and weakest.

4. Discussion

4.1. Part 1 – REINFORCE and Actor-Critic

Training instability in REINFORCE. All the different REINFORCE versions show very noisy optimization, which is common for Monte Carlo policy gradients in long-horizon tasks. Episodic returns lead to high-variance gradient estimates and without constraints (unlike TRPO [8] or PPO [9]), vanilla REINFORCE can be at risk of destructive updates that cause notable performance collapses.

Effect of the baseline. Subtracting a baseline reduces variance without altering the expectation [10]. The experiments in Fig. 1a suggest that baseline quality strongly affects optimization behaviour. Without a baseline ($b=0$), high-variance returns lead to slow and unstable learning. A higher baseline ($b=20$) helps to speed up the initial optimization process but, over time, it starts to be misaligned with the reward scale, which leads to unstable updates. The adaptive baseline ($b=P_{25}$) tracks recent performance and keeps a more informative advantage signal during training. This could help explain why we see improvement after a short drop in performance.

Limitations of REINFORCE for hopping. All the proposed baselines learn to stand and maintain their balance, but none of them manage to hop forward. The length of the episodes plateau at the maximum timesteps without reaching the higher reward scores common to agents capable of hopping. A plausible explanation is a relatively easy local optimum that mainly provides reward through the healthy bonus. In contrast, efficient hopping requires more coordinated control that is harder for high-variance Monte Carlo methods.

Actor-Critic slow convergence. Convergence is slower compared to any REINFORCE variant because the critic must first learn a reliable value function. Additionally, a lower learning rate was used for Actor-Critic; higher rates provided poor results. This is due to the update frequency: REINFORCE averages gradients over an entire episode before updating, reducing the impact of noisy single timestep

signals, while Actor-Critic updates at every timestep, making it more sensitive to noise.

Exploration vs exploitation. Actor-Critic explores much more than REINFORCE variants, becoming less prone to stalling in local minima. By computing the actor gradient (in equation 3) with respect to σ , we see that: if the action was "sub optimal" ($|a - \mu| > \sigma$) but good (a positive TD or cumulative reward) σ rises. Otherwise it is lowered. If the action is "optimal" ($|a - \mu| < \sigma$) the opposite logic is used. In a local minima most of the optimal actions will lead to improvement, thus most updates will be in favour of reducing σ even if a couple of exploratory actions bring improvements. In reinforce the average cancels the contribute of the exploratory actions, thus the update lowers the std.

Limitations of Actor Critic After reaching the peak cumulative reward the model goes back to the sub optimal policy. This is likely due to on-policy instability: when exploration is high, the model may generate a sub optimal action that leads it in a worse state, from which it generates bad data that reinforces bad behaviour.

4.2. Part 2 – PPO, SAC, and DR

PPO vs. SAC sample efficiency. The high success rate observed during evaluation can largely be attributed to the entropy-regularized objective used by SAC, which promotes exploration and helps the agent avoid prematurely converging to sub-optimal policies. Such properties are particularly important in the PushTask environment, where the cube’s position is randomized at the beginning of each episode. As a consequence, the agent must continuously adapt its behaviour and orientation in order to push the cube towards the target location from different initial configurations. Empirical results strongly support this observation: PPO does not exhibit clear convergence even after 10^7 timesteps. Even when trained with a surrogate reward system, where the agent is rewarded for moving the arm closer to the cube, PPO still does not learn a pushing policy due to the fact that pushing in the wrong direction reduces the episodic return, and thus stagnates in a sub-optimal policy that does not move the cube.

Domain Randomization As already mentioned, the most relevant metric to consider should be the mean return, rather than the success rate. Most domain randomized training strategies proved to perform better than the baseline on that front. The target-trained policy moves the cube more decisively, trading control over the cube for higher return due to the shorter episode length: randomization helps the agent in developing that decisiveness without relying on target environment priors during training. The best results achieved by UDR comes with the cost of manually selecting said range, unfeasible for more complex tasks.

The ADR trained model has the highest standard deviation among all methods and is the only one to perform strictly

worse than the baseline. A possible explanation could be that the upper boundary expands well past a plausible range, reaching $[0.001, 27.6]$ kg. This, along with the relatively high boundary sampling value $p_b = 0.8$, makes it so that the agent mainly trains with masses at the extremes of the current interval. Even though, at some point, the model proves to be able to handle masses closer to the target one, after boundary expansion it eventually stops seeing them often enough to crystallize an optimal policy that could generalize on a likely target environment. We attempted to restrict boundary expansion by enforcing stricter performance thresholds τ and reducing the boundary sampling probability p_b : this resulted in poorer performance, proving ADR is very sensitive to hyperparameter tuning.

BADR remedies to ADR’s failure, but does so by reintroducing a prior on the range. This result raises the question: *what is the advantage of BADR over UDR?* Both strategies ultimately sample from the same distribution whenever BADR saturates the range. We expect BADR to significantly outperform UDR in settings where more parameters undergo randomization: UDR would waste early training stages on very different configurations causing conflicting updates, while BADR curriculum would expand safely from a stable foundation, without the risk of excessive drift, allowing for even greater boundaries.

Limitations of the experiment. The robustness of the baseline model is remarkable and can be attributed to the similarity between the two environments. A 4 kg mass shift does not fundamentally change the task at hand: it simply makes the moving process slower due to the higher inertia of the cube. In contrast, real-world environments introduce many additional uncertainties (variation in surface friction, sensor noise, imperfections in object geometry, etc.). To truly simulate a reality gap the target environment should have introduced more differences that should have been addressed by domain randomization strategies.

5. Conclusion

In this work we looked into using reinforcement learning for robotic control and sim-to-real transfer through domain randomization. On Hopper-v4, Actor-Critic surpassed all REINFORCE versions, showing effective hopping behaviour. On PandaGym PushTask, SAC proved to be better suited for the task than PPO thanks to its entropy-centered learning. Domain Randomization closed most of the sim-to-sim gap, but not all strategies were beneficial: UDR proved to be the most effective, our BADR variant performed similarly but ADR drifted to a meaningless mass range. Limitations include the single-parameter randomization and the simplified sim-to-sim setting. Future work should extend to larger sim-to-sim gaps and multi-parameter randomization such as friction and contact.

6. Acknowledgments

The authors declare that they **did not** use any Generative AI tools to write this project report.

Regarding the project code, *Claude Sonnet 4.6* (Anthropic) was used to help with environment integration and configuration in Kaggle/Google Colab. This was mainly to fix problems connected to virtual environments and dependencies. It was also used for code related to creating graphs and for the setup and configuration of Weights & Biases (wandb). Its use was limited to practical support and did not contribute with applying reinforcement learning concepts, methodological choices, or experimental design.

References

- [1] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. Openai gym. arXiv preprint arXiv:1606.01540, 2016. [1](#)
- [2] Quentin Gallouédec, Nicolas Cazin, Emmanuel Dellandréa, and Liming Chen. panda-gym: Open-source goal-conditioned environments for robotic learning, 2021. [1](#)
- [3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proc. ICML*, 2018. [1](#), [2](#)
- [4] S. Höfer et al. Perspectives on sim2real transfer for robotics: A summary of the r:ss 2020 workshop. arXiv preprint arXiv:2012.03806, 2020. [1](#)
- [5] V. R. Konda and J. N. Tsitsiklis. Actor-critic algorithms. In *Adv. Neural Inf. Process. Syst.*, 2000. [1](#)
- [6] OpenAI et al. Solving rubik’s cube with a robot hand. arXiv preprint arXiv:1910.07113, 2019. [1](#), [3](#)
- [7] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. [1](#), [2](#)
- [8] J. Schulman, S. Levine, P. Moritz, M. Jordan, and P. Abbeel. Trust region policy optimization. In *Proc. ICML*, 2015. [4](#)
- [9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017. [1](#), [2](#), [4](#)
- [10] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018. [1](#), [2](#), [4](#)
- [11] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Adv. Neural Inf. Process. Syst.*, 2000. [2](#)
- [12] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *Proc. IEEE/RSJ IROS*, 2017. [1](#)
- [13] E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. In *Proc. IEEE/RSJ IROS*, 2012. [1](#)
- [14] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 1992. [1](#), [2](#)